

A Mini Project Report on

AI-BASED INTELLIGENT ONLINE EXAM PROCTORING SYSTEM

submitted in partial fulfillment for award of the degree of Bachelor of Technology in
Computer Science and Engineering (AI&ML)

By

A.Harshitha	23321A6613
VJS Pranathi	23321A6620
B.Akshitha	23321A6602

Under the esteemed Guidance of

Internal Guide

E.Drakshavalli

Assistant Professor, CSE(AI&ML)



Bhoj Reddy Engineering College for Women
Department of Computer Science and Engineering (AI&ML)

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade,

Approved by AICTE and Affiliated to JNTUH)

Recognised by UGC under section 2(f) of the UGC Act, 1956.

Vinaynagar, IS Sadan Crossroads, Hyderabad – 500 059

Ph: +91-40-2453 7282; Website: www.brecw.ac.in; Email: principal@brecw.ac.in

2025- 2026



Bhoj Reddy Engineering College for Women

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade, Approved by AICTE and Affiliated to JNTUH)

Recognised by UGC under section 2(f) of the UGC Act, 1956.

Vinaynagar, I S Sadan Crossroads, Hyderabad 500 059, Telangana

Telephone: 040-2453 7282, Website: www.brecw.ac.in, Email: principal@brecw.ac.in



Date: 30-03-2026

CERTIFICATE

This is to certify that the Mini Project Report entitled “**AI-BASED INTELLIGENT ONLINE EXAM PROCTORING SYSTEM**” is a bonafide work carried out by

A.Harshitha

23321A6613

VJS Pranathi

23321A6620

B.Akshitha

23321A6602

in partial fulfillment for award of the degree of Bachelor of Technology in **Computer Science and Engineering(AI&ML)** department from Bhoj Reddy Engineering College for Women, Hyderabad, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH).

Internal Guide

E.Drakshavalli

Head of the Department

Dr M Seshu Bhavani

External Examiner

ACKNOWLEDGMENT

It is our pleasure to express our wholehearted thanks to Dr M Seshu Bhavani Associate Professor & Head of Computer Science and Engineering (AI&ML) Department for providing the necessary facilities to undergo this project.

We are thankful to our project coordinator Ms Sameera Begum, Assistant Professor, Computer Science and Engineering (AI&ML) Department, for giving us this opportunity to do this project.

We are also thankful to our internal guide Ms E.Drakshavalli, Associate Professor, Computer Science and Engineering (AI&ML) Department for her extreme guidance and support in completing this project successfully.

We are also thankful to the staff members of Computer Science and Engineering (AI&ML) Department, my friends and to our parents who helped us in completing this project successfully.

By

A.Harshitha (23321A6613)

VJS Pranathi (23321A6620)

B.Akshitha (23321A6602)

CONTENTS

List of figures

List Of tables

Abstract

1.Introduction	1-3
1.1 Introduction	2
1.2 Scope	2
1.3 Existing System	2
1.3.1 Problems in Existing System	2
1.4 Proposed System	3
1.4.1 Advantages of Proposed System	3
2. Requirement Analysis	4-6
2.1 Functional Requirements	5
2.1.1 Student Module	5
2.1.2 Proctor Module	5
2.1.3 Admin Module	5
2.2 Non-functional Requirements	5
2.2.1 Performance	5
2.2.2 Scalability	5
2.2.3 Reliability	5
2.2.4 Security & Privacy	5
2.2.5 Usability	5
2.2.6 Maintability	5
2.2.7 Platform Independence	5
2.3 Computation Resource Requirement	6
2.3.1 Hardware Requirement	6

2.3.2 Software Requirement	6
3.Design	7-12
3.1 Architecture	8
3.1.1 Software Architecture	8
3.1.2 Technical Architecture	8-9
3.2 UML Diagrams	9-12
3.2.1 Use Case Diagram	9
3.2.2 Class Diagram	10
3.2.3 Sequence Diagram	10-11
3.2.4 Activity Diagram	12
4. Implementation	13-21
4.1 Technologies and Libraries used	14-15
4.1.1 Python	14
4.1.2 OpenCV	14
4.1.3 Flask	14
4.1.4 HTML	14
4.1.5 CSS	14
4.1.6 SQLite	14
4.1.7 GitHub	14
4.1.8 OpenCV(cv2)	14
4.1.9 NumPy	14-15
4.1.10 Flask	15
4.1.11 Imutils	15
4.1.12 Pre-trained Models Used	15
4.2 Data PreProcessing	15-17
4.2.1 Video Frame Capture	15
4.2.2 Frame Resizing	15-16

4.2.3 Blob Conversion	16
4.2.4 Face Detection Input Preparation	16
4.2.5 Face Coordinate Extraction	16
4.2.6 Feature Calculation	16
4.2.7 Threshold Comparision	17
4.3 Model Implementation	17-18
4.4 Web Application Frontend	18
4.5 Flask Backend & Routes	18-19
4.6 System Deployment	19-20
4.7 Summary Of Component Integration	20
4.8 Pseudo Code	20-21
5. Testing	22-25
5.1 Overview	23
5.2 Test Objectives	23
5.3 Stages Of Testing	23-24
5.3.1 Unit Testing	24
5.3.2 Integration Testing	24
5.3.3 Functional Testing	24
5.3.4 System Testing	24
5.4 Types Of Testing	25
5.4.1 White Box Testing	24
5.4.2 Black Box Testing	25
5.5 Test Cases	25
6. ScreenShots	26-27

7.Conclusion And Future Enhancements	28-29
8.References	30-31

LIST OF FIGURES

Figure No	Description	PageNo
Figure 1	System Architecture	8
Figure 2	Technical Architecture	8
Figure 3	Use Case Diagram for cursor movements	9
Figure 4	Class Diagram for Cursor movements	10
Figure 5	Activity Diagram for Cursor Movements	11
Figure 6	Sequence Diagram for Cursor Movements	12
Figure 7	Webcam Section	27
Figure 8	Proctor Section	27

LIST OF TABLES

Table No	Description	Page No
Table-1	Summary Of Component Integration	20
Table-2	Functional Testing	24
Table-3	Test Cases	25

ABSTRACT

The rapid growth of online education has increased the use of online examinations. However, maintaining fairness and preventing malpractice during online exams is a major challenge because students are not physically monitored by invigilators. Traditional methods such as manual monitoring through video calls are time-consuming and difficult to manage when a large number of students are involved.

This project presents an AI-Based Intelligent Online Exam Proctoring System that helps automate the monitoring process using computer vision techniques. The system uses a webcam to observe the student during the examination and detect suspicious activities such as absence of face, presence of multiple faces, or unusual movements. Based on these observations, a cheating score is calculated to identify potential malpractice.

The results of the monitoring process are stored in a database, and a web-based dashboard allows the proctor to view and analyze the exam records. This system reduces human effort, improves monitoring efficiency, and provides a scalable solution for maintaining academic integrity in online examinations.

1.INTRODUCTION

1.INTRODUCTION

1.1 INTRODUCTION OF THE PROJECT

Online examinations have become increasingly popular due to the growth of digital education. However, maintaining fairness and preventing malpractice during online exams is a major challenge. Traditional online exam systems rely on manual monitoring by human proctors, which is inefficient and error-prone.

To address these issues, this project proposes an AI-Based Intelligent Online Exam Proctoring System that automatically monitors students using computer vision techniques and identifies suspicious behavior during examinations

1.2 SCOPE OF THE PROJECT

The scope of the proposed system includes real-time monitoring of students through webcams to ensure exam integrity. The system automatically detects suspicious activities such as absence of face, presence of multiple faces, and frequent looking away from the screen.

It generates a cheating score and stores exam results in a database for proctor review. The system reduces manual supervision and can be extended in the future to include eye tracking, object detection, and advanced analytics

1.3 EXISTING SYSTEM

In the existing online examination system, proctors manually monitor students through live video feeds. The effectiveness of monitoring depends entirely on human attention.

Such systems lack automated cheating detection, are difficult to scale for large numbers of students, and do not provide structured reports for later analysis

1.3.1 PROBLEMS IN EXISTING SYSTEM

- High dependency on human proctors
- Time-consuming and labor-intensive
- Limited scalability
- Prone to human error
- No automatic detection of cheating behavior
- No structured evidence or exam logs

1.4 PROPOSED SYSTEM

The proposed system uses artificial intelligence and computer vision techniques to automatically monitor students during online examinations.

A pre-trained face detection model detects student presence. Rule-based behavior analysis is applied to identify suspicious activities such as no face detected, multiple faces detected, and looking away from the camera.

The system calculates a cheating score and classifies the exam session as Normal, Suspicious, or Cheating, and stores results in a database for proctor review

1.4.1 ADVANTAGES IN PROPOSED SYSTEM

- Automated cheating detection
- Reduced human effort
- Real-time monitoring
- Scalable for large exams
- Consistent and unbiased evaluation
- Secure storage of exam results

2.REQUIREMENT ANALYSIS

2.REQUIREMENT ANALYSIS

2.1FUNCTIONAL REQUIREMENTS

1. **Student Module** : Enables students to securely attend online exams with real-time AI monitoring and result submission.
2. **Proctor Module** : Allows proctors to monitor exams live, review suspicious activities, and access cheating reports.
3. **Admin Module** : Manages users, schedules exams, configures system settings, and oversees overall reports

2.2NON-FUNCTIONAL REQUIREMENTS

- **Performance** – The system should monitor students in real-time with minimal delay (within 2 seconds).
- **Scalability** – The system should handle a large number of students simultaneously.
- **Reliability** – The system should operate continuously without crashes during exams.
- **Security & Privacy** – The system should protect student data and follow privacy policies.
- **Usability** – The system should provide an easy and user-friendly interface.
- **Maintainability** – The system should be easy to update and extend with new features.
- **Platform Independence** – The system should work across different operating systems

2.3 COMPUTATION RESOURCE REQUIREMENT

2.3.1 HARDWARE REQUIREMENT

- **Processor:** Intel i5 or above
- **RAM:** 8 GB
- **Webcam**
- **Storage:** 100 GB

2.3.2 SOFTWARE REQUIREMENT

- **Operating System**
 - Windows 10
- **Programming Language**
 - Python 3.8+ for scripting and model development.
- **Frontend Technologies**
 - HTML
 - CSS
- **Backend Technologies**
 - Flask
- **Database**
 - SQLite
- **Libraries**
 - OpenCV
 - Dlib
 - SQLite
- **IDE**
 - VS Code / PyCharm

3.DESIGN

3.DESIGN

3.1 ARCHITECTURE

3.1.1 SOFTWARE ARCHITECTURE

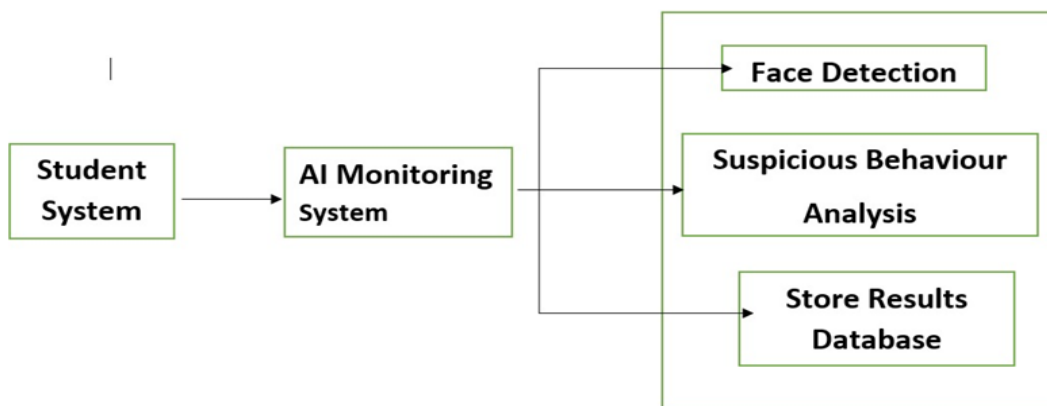


Figure-1

3.1.2 TECHNICAL ARCHITECTURE

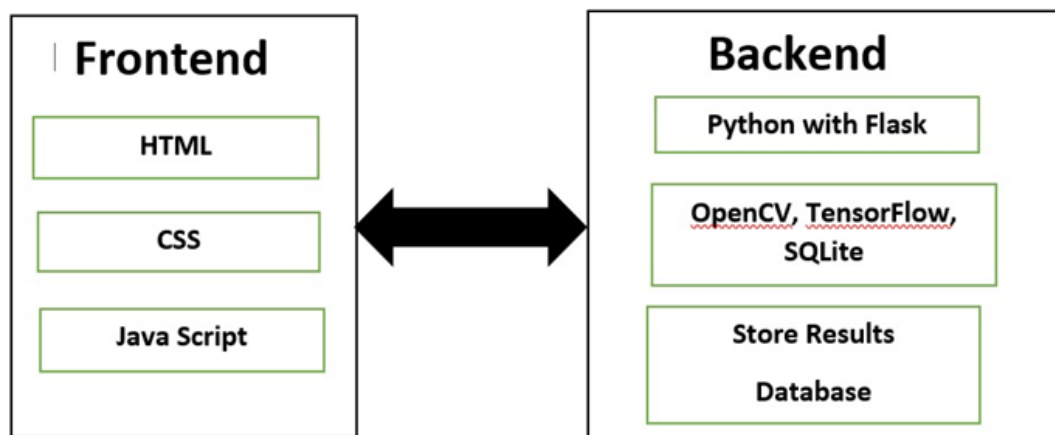


Figure-2

In the fig 3.1.2, computers, software or networks, the overall design of a computing system and the logical and physical interrelationships between its components. A system architecture is the conceptual model that defines the structure, behaviour, and more views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structures and behaviours of the system. A system architecture can consist of system components and the sub-systems developed that will work together to implement the overall system. There have been efforts to formalize languages to describe system architecture, collectively these are called architecture description languages(ADLS).

3.2 UML DIAGRAMS

UML stands for Unified Modelling Language. UML is a standardized general-purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Metamodel and a notation. The Unified Modelling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modelling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems.

3.2.1 USE CASE DIAGRAM

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the system. The main purpose of a use case diagram is to show what system functions are performed for which actor roles of the actors in the system can be depicted.

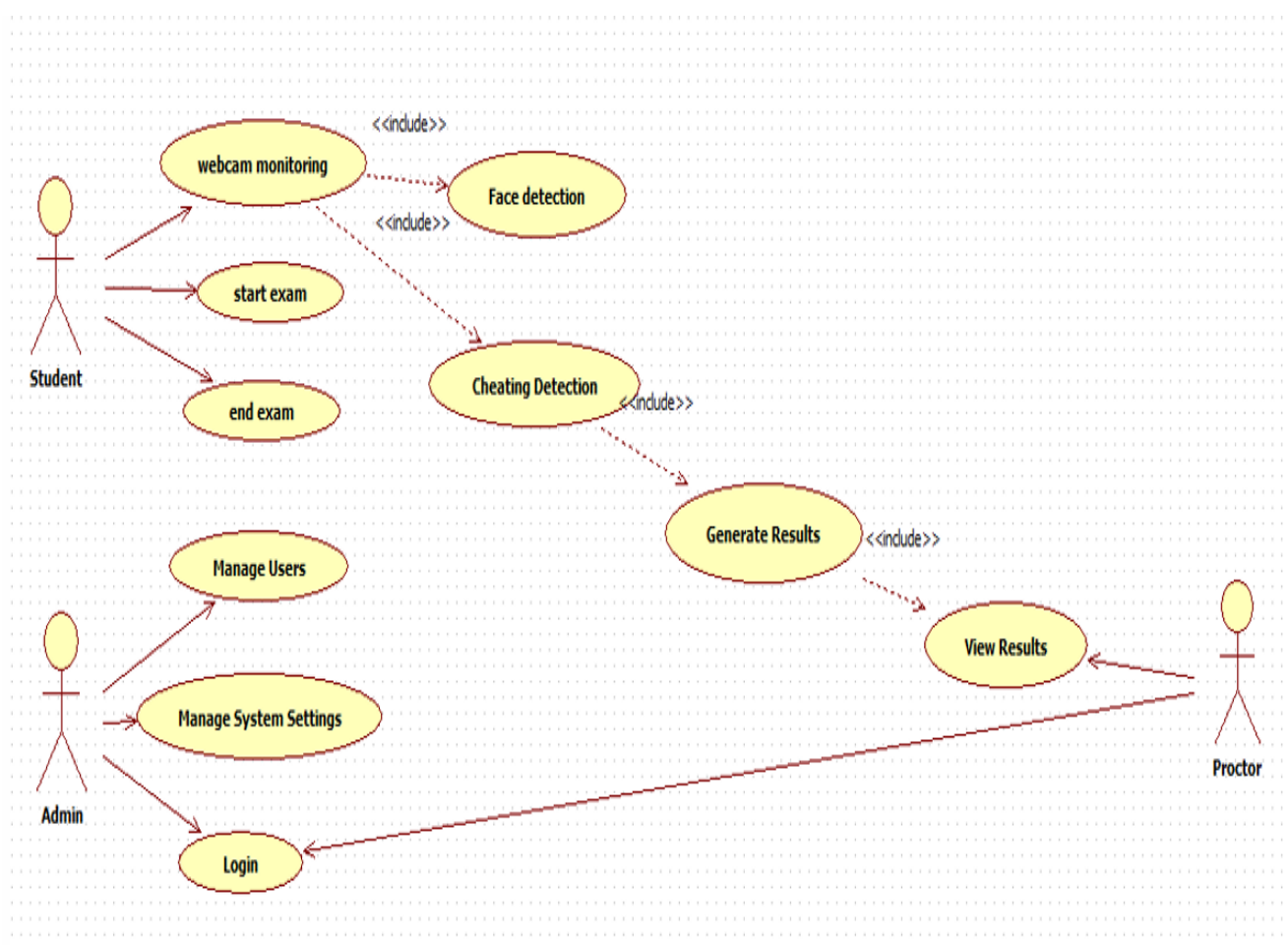


Figure-3

3.2.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

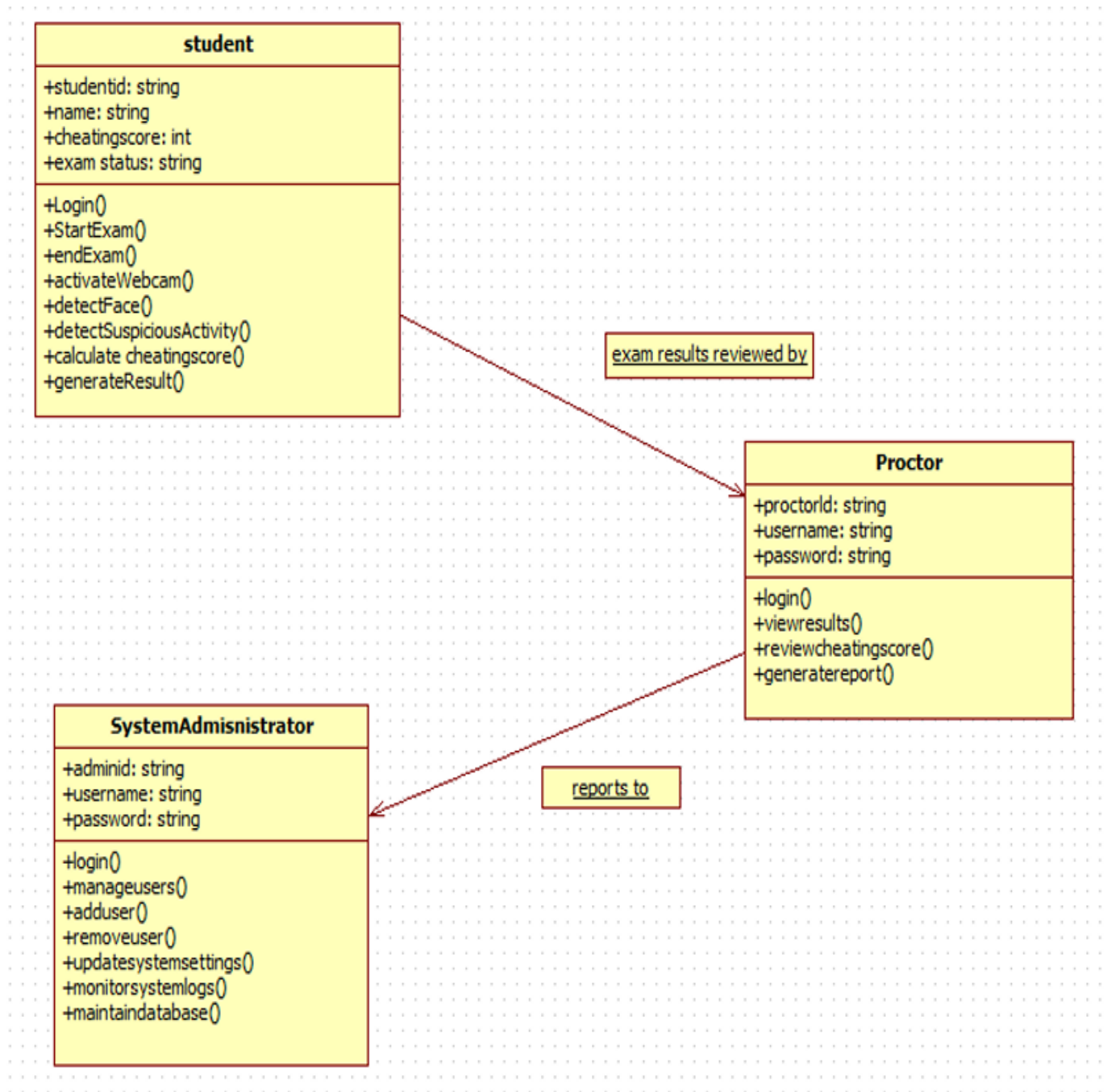
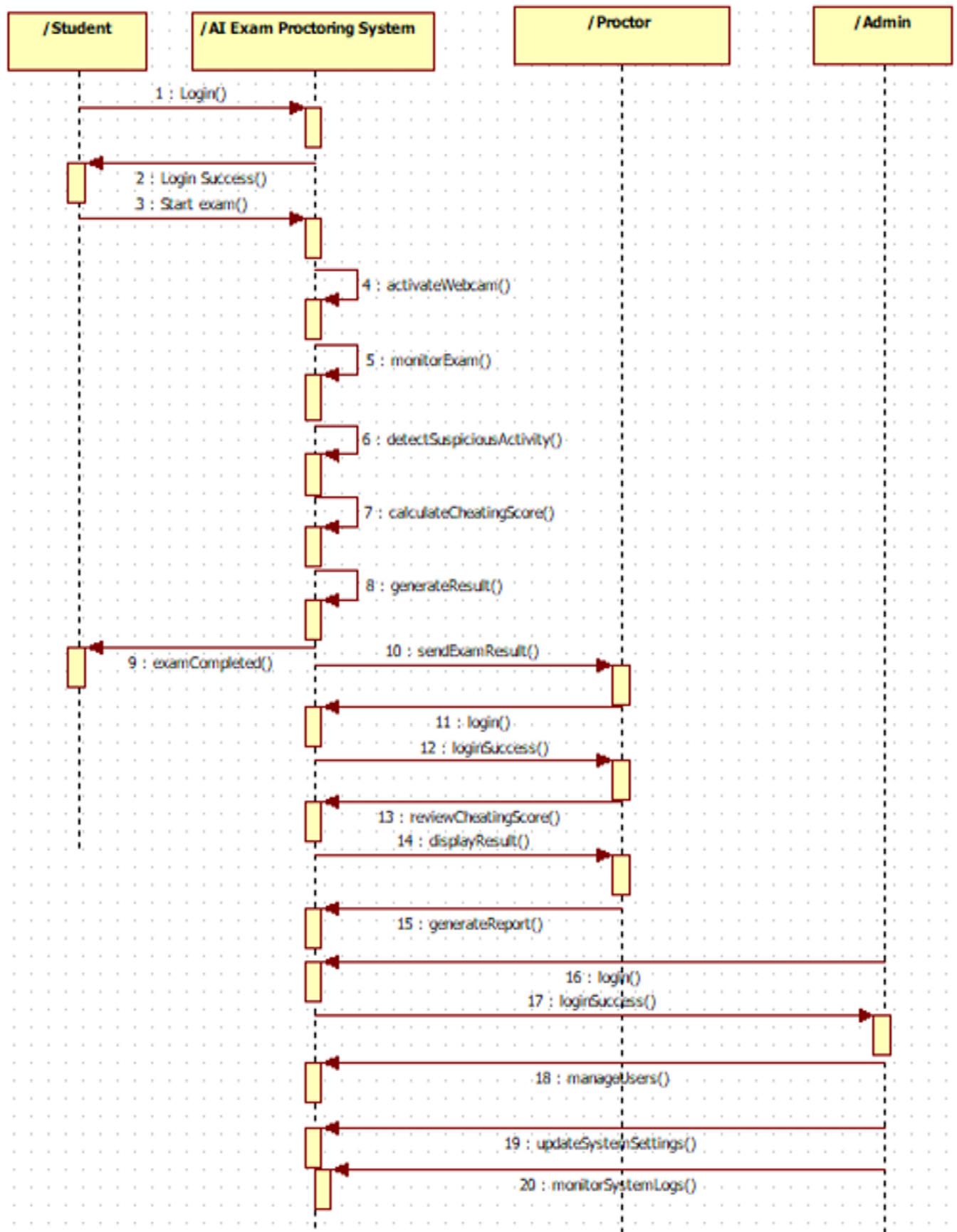
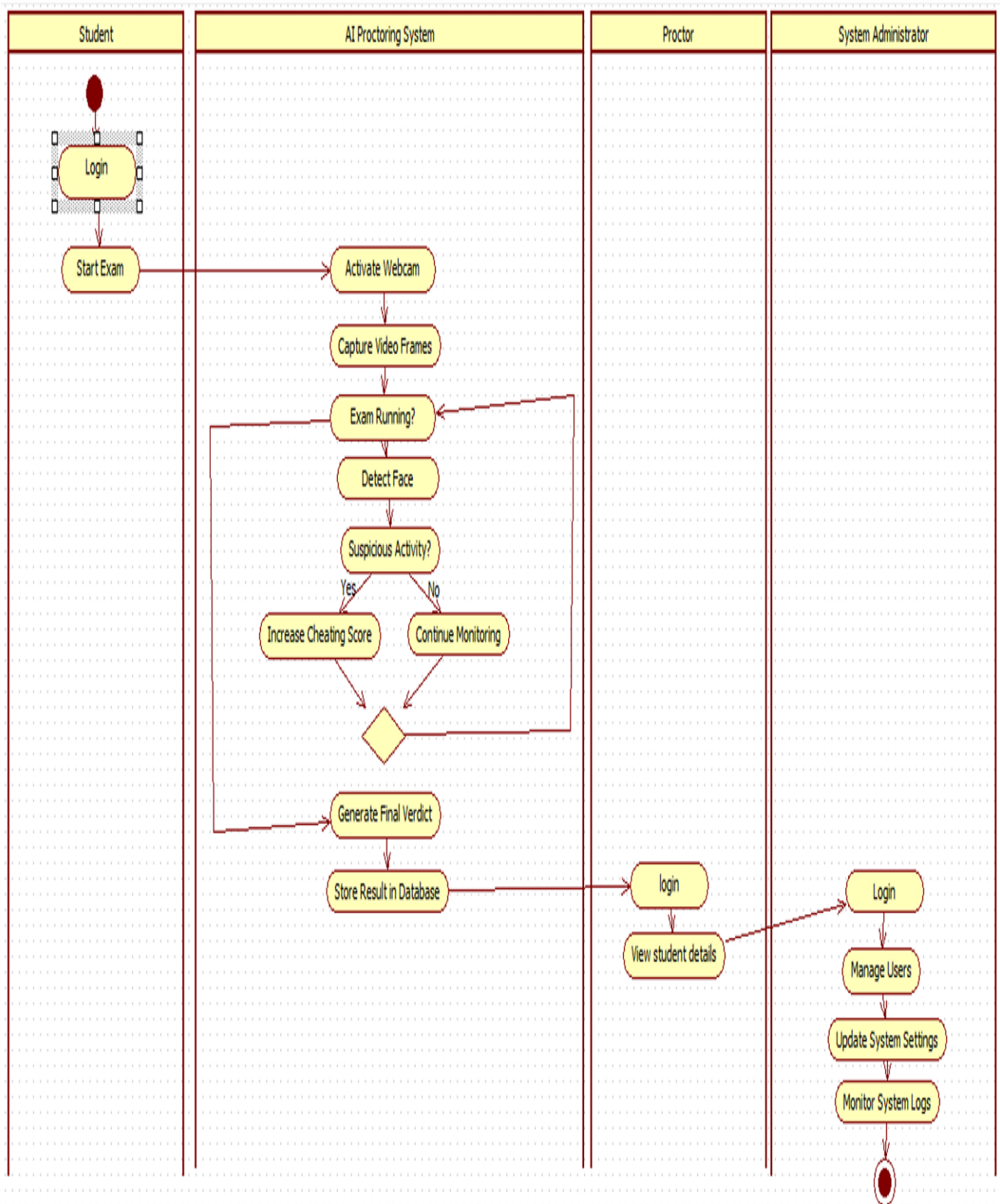


Figure-4.

3.2.3 SEQUENCE DIAGRAM

Sequence Diagrams Represent the objects participating the interaction horizontally and time vertically.

**Figure-5**

3.2.4 ACTIVITY DIAGRAM**Figure-6**

4.IMPLEMENTATION

4.IMPLEMENTATION

4.1 TECHNOLOGIES AND LIBRARIES USED:

4.1.1 Python

- Main programming language used to develop the system.
- Handles face detection logic, cheating score calculation, and backend processing.

4.1.2 OpenCV

- Open-source computer vision library.
- Used for real-time webcam access, face detection, and image processing.

4.1.3 Flask

- Lightweight Python web framework.
- Used to build the proctor dashboard and backend server.

4.1.4 HTML

- Used to design the web pages of the proctor dashboard.

4.1.5 CSS

- Used for styling the dashboard interface.

4.1.6 SQLite

- Lightweight database used to store cheating results and student data.

4.1.7 Git & GitHub

- Used for version control and project repository management.

4.1.8 OpenCV (cv2)

Used for:

- Face detection
- Video capture
- Image processing

Example in code:

```
import cv2
```

4.1.9 NumPy

Used for:

- Numerical operations
- Image matrix processing

Example:

```
import numpy as np
```

4.1.10 Flask

Used for:

- Creating the backend server
- Connecting dashboard with database

Example:

```
from flask import Flask
```

4.1.11 Imutils

Used for:

- Image processing utilities
- Simplifying OpenCV functions

Example:

```
import imutils
```

4.1.12 Pre-trained Models Used

Your project also uses pre-trained deep learning models for face detection:

- res10_300x300_ssd_iter_140000.caffemodel
- deploy.prototxt

These are used by OpenCV DNN module.

4.2 DATA PREPROCESSING

4.2.1. Video Frame Capture:

The system captures live video frames from the student's webcam using OpenCV.

Example:

```
ret, frame = cap.read()
```

This converts the webcam stream into individual frames for processing.

4.2.2. Frame Resizing:

Frames are resized to a fixed dimension so that the detection model works efficiently and consistently.

Example:

```
cv2.resize(frame, (300, 300))
```

This reduces computational cost and improves performance.

4.2.3. Blob Conversion (Image Normalization):

The frame is converted into a blob format using OpenCV's DNN module before passing it to the face detection model.

Example:

```
blob = cv2.dnn.blobFromImage(frame, 1.0, (300,300),  
(104.0,177.0,123.0))
```

This step normalizes pixel values and prepares the image for the neural network.

4.2.4. Face Detection Input Preparation:

The processed frame is fed into the pretrained face detection model.

Example:

```
model.setInput(blob)  
  
detections = model.forward()
```

This identifies face coordinates in the frame.

4.2.5. Face Coordinate Extraction:

The system extracts bounding box coordinates of detected faces.

Example:

```
(x, y, x1, y1) = box.astype("int")
```

These coordinates help determine the position of the face relative to the frame center.

4.2.6. Feature Calculation (Face Center Position):

The center of the detected face is calculated to analyze head movement.

Example:

```
face_center_x = (x + x1) // 2  
  
face_center_y = (y + y1) // 2
```

This step is used to detect whether the student is looking away from the screen.

4.2.7. Threshold Comparison:

The face position is compared with the frame center using predefined thresholds.

If the deviation exceeds the threshold, the system increases the cheating score.

4.3 MODEL IMPLEMENTATION

The proposed AI-based Online Exam Proctoring System uses computer vision techniques to monitor student behavior during an online examination. The system detects faces using a pre-trained deep learning model provided by OpenCV's DNN module and analyzes the position of the face relative to the screen center.

Instead of training a custom machine learning model, the system utilizes a **pre-trained face detection model (SSD-based ResNet architecture)** to identify faces in real time from the webcam feed. The detected face coordinates are used to calculate the center of the face and compare it with the frame center.

Based on predefined thresholds, the system determines whether the student is looking away from the screen or behaving suspiciously. If suspicious behavior is detected, the cheating score increases and the system updates the status accordingly.

The model also detects conditions such as:

- No face detected
- Multiple faces detected
- Face movement beyond the threshold

These conditions help identify potential cheating behavior during the examination.

Sample Code for Face Detection and Monitoring

```
def detect_cheating(frame, faces, frame_w,
                    frame_h):
    cheating_score = 0
    status = "NORMAL"

    if len(faces) == 0:
        cheating_score += 1
        status = "NO FACE DETECTED"

    elif len(faces) > 1:
        cheating_score += 5
        status = "MULTIPLE FACES DETECTED"

    else:
        (x, y, x1, y1) = faces[0]

        face_center_x = (x + x1) // 2
        face_center_y = (y + y1) // 2

        frame_center_x = frame_w // 2
        frame_center_y = frame_h // 2

        threshold_x = int(frame_w * 0.10)
        threshold_y = int(frame_h * 0.10)
```

```
        if abs(face_center_x -
frame_center_x) > threshold_x:
            cheating_score += 2
            status = "LOOKING LEFT OR
RIGHT"

        elif abs(face_center_y -
frame_center_y) > threshold_y:
            cheating_score += 2
            status = "LOOKING UP OR DOWN"

    return status, cheating_score
```

This rule-based logic helps the system detect suspicious behavior in real time.

4.4 WEB APPLICATION FRONTEND

The web application frontend provides an interface for the proctor to monitor student behavior during the examination. The dashboard allows the proctor to view cheating results generated by the AI monitoring system.

The frontend is developed using **HTML, CSS, and basic web technologies** to create a simple and responsive user interface.

The web interface includes the following components:

- **Proctor Login Page** – allows the proctor to securely access the monitoring dashboard.
- **Dashboard Interface** – displays student monitoring results.
- **Cheating Score Display** – shows the calculated cheating score for each student.
- **Status Indicator** – displays the behavior classification such as Normal, No Face Detected, or Multiple Faces Detected.
- **Timestamp Logging** – records the time when suspicious activity was detected.

The frontend communicates with the backend server built using **Flask**, which retrieves monitoring results from the database and displays them on the dashboard.

This interface enables the proctor to easily monitor exam activity and identify suspicious behavior during online examinations.

4.5 FLASK BACKEND & ROUTES

The backend of the AI Online Exam Proctoring System is developed using the Flask web framework. Flask handles communication between the AI monitoring module and the web dashboard used by the proctor.

The backend manages data processing, database communication, and routing of web pages.

The following routes are implemented in the system:

- **/** : Displays the login page for the proctor.

- **/login** : Authenticates the proctor credentials.
- **/dashboard** : Displays monitoring results such as student ID, cheating score, verdict, and timestamp.
- **/results** : Fetches monitoring data stored in the database and sends it to the dashboard interface.

Sample Flask Route:

```
from flask import Flask, render_template
import sqlite3

app = Flask(__name__)

@app.route("/")
def login():
    return render_template("login.html")

@app.route("/dashboard")
def dashboard():
    conn = sqlite3.connect("database.db")
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM
exam_results")
    data = cursor.fetchall()

    conn.close()

    return
render_template("dashboard.html",
results=data)

if __name__ == "__main__":
    app.run(debug=True)
```

This backend allows the proctor to view real-time monitoring results generated by the AI system.

4.6 SYSTEM DEPLOYMENT

The AI Online Exam Proctoring System can be deployed in different environments depending on the application requirements.

- **Local Deployment:**
The system can be executed locally on a computer using Python and Flask. The AI module runs on the student system, and the dashboard is accessed through a web browser.
- **Server Deployment:**
The Flask backend can be deployed on a remote server so that proctors can monitor students from different locations.
- **Cloud Deployment:**
The application can also be deployed on cloud platforms such as AWS, Google Cloud, or Heroku for large-scale monitoring.

- **Database Integration:**

SQLite is used for storing monitoring results including student ID, cheating score, verdict, and timestamps.

4.7 SUMMARY OF COMPONENT INTEGRATION

Component	Function
OpenCV	Captures webcam video and performs face detection
Pre-trained Face Detection Model	Detects faces using deep learning model
AI Monitoring Logic	Calculates Cheating Score based on face position and behavior
Python Backend	Processes monitoring results
Flask Framework	Connects backend with the web dashboard
SQLite Database	Stores monitoring results and timestamps
HTML\CSS Frontend	Displays monitoring results in the proctor dashboard

Table-1

4.8 PSEUDO CODE:

Initialize webcam

Load face detection model

cheating_score = 0

while exam_running:

 capture frame from webcam

 detect faces in frame

 if no face detected:

 cheating_score += 1

 elif multiple faces detected:

 cheating_score += 5

```
else:
    check face position
    if looking away:
        cheating_score += 2
display cheating score
if exam ends:
    calculate final verdict
    store result in SQLite database
```

5.TESTING

5. TESTING

5.1 OVERVIEW:

The AI Online Exam Proctoring System was tested at multiple levels to ensure that it performs correctly and reliably during real-time exam monitoring.

The testing process focused on validating the following aspects:

- Face detection accuracy
- Cheating score calculation logic
- Real-time video processing performance
- Web dashboard functionality
- Database storage and retrieval of monitoring results

These tests ensured that the system correctly identifies suspicious behaviour and displays the results on the proctor dashboard.

5.2 TEST OBJECTIVES

The main objectives of testing the system were:

- Ensure correct detection of student faces using the webcam
- Verify detection of suspicious behaviour such as looking away from the screen
- Validate cheating score calculation based on predefined rules
- Test database storage of monitoring results
- Ensure the proctor dashboard displays correct monitoring information.

5.3 STAGES OF TESTING

5.3.1 UNIT TESTING

Each module of the system was tested individually to ensure proper functionality.

The following components were tested separately:

- Face detection module using OpenCV
- Cheating score calculation logic
- Database insertion and retrieval functions
- Web dashboard rendering

Test Case Example:

- **Input:** Student moves face away from screen center

- **Expected Output:** Status changes to "Looking Away" and cheating score increases.

5.3.2 INTEGRATION TESTING

Integration testing was performed to ensure that different modules of the system work together correctly.

The following integrations were tested:

- OpenCV face detection with cheating detection logic
- Python monitoring module with database storage
- Flask backend with web dashboard frontend

This ensured smooth data flow between the monitoring system and the proctor dashboard.

5.3.3 FUNCTIONAL TESTING

All functionalities were validated according to the system requirements.

Module	Input
Face Detection	Webcam Frame
Cheating Detection	Face movement away from center
Database Module	Monitoring result
Flask Web App	Dashboard request

Table-2

5.3.4 SYSTEM TESTING

The complete system was tested to verify overall functionality and performance.

The following tests were conducted:

- Monitoring student behaviour through webcam
- Detecting suspicious head movement
- Testing system performance during continuous video processing
- Verifying dashboard display of cheating results

5.4 TYPES OF TESTING

5.4.1 WHITE BOX TESTING

- White box testing was used to verify the internal logic of the cheating detection algorithm and face detection implementation.
- The internal conditions such as face position thresholds and cheating score calculations were tested to ensure correct behaviour detection.

5.4.2 BLACK BOX TESTING

Black box testing was performed by evaluating the system based on input and output behaviour without analyzing internal code.

Different scenarios were tested such as:

- No face detected
- Multiple faces detected
- Student looking away from screen

The system correctly detected these conditions and updated the cheating score accordingly.

5.5 TEST CASES

Tested	Test Name	Inputs	Expected output	Status
1.	Start Webcam	Camera input	Webcam activated	Success
2.	Face Detection	Student Face	Face Detected	Success
3.	No Face	Empty Frame	Increase Cheating Score	Success
4.	Multiple Faces	Two faces	Increase Cheating Score	Success
5.	Looking Away	Face not Centred	Increase Cheating score	Success
6.	Store Results	Student ID	Save Result in Database	Success

Table-3

6. SCREENSHOTS

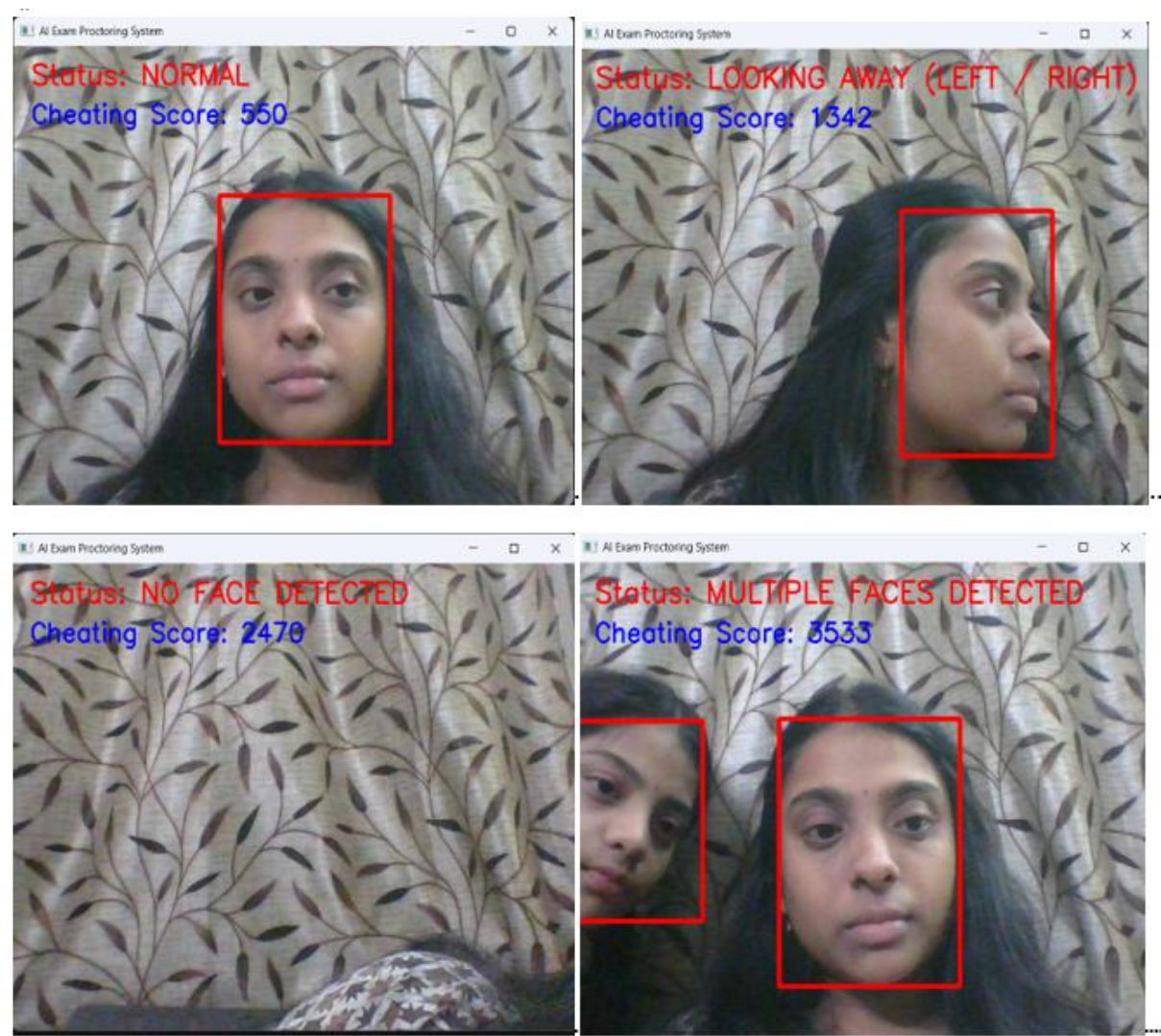


Figure-7

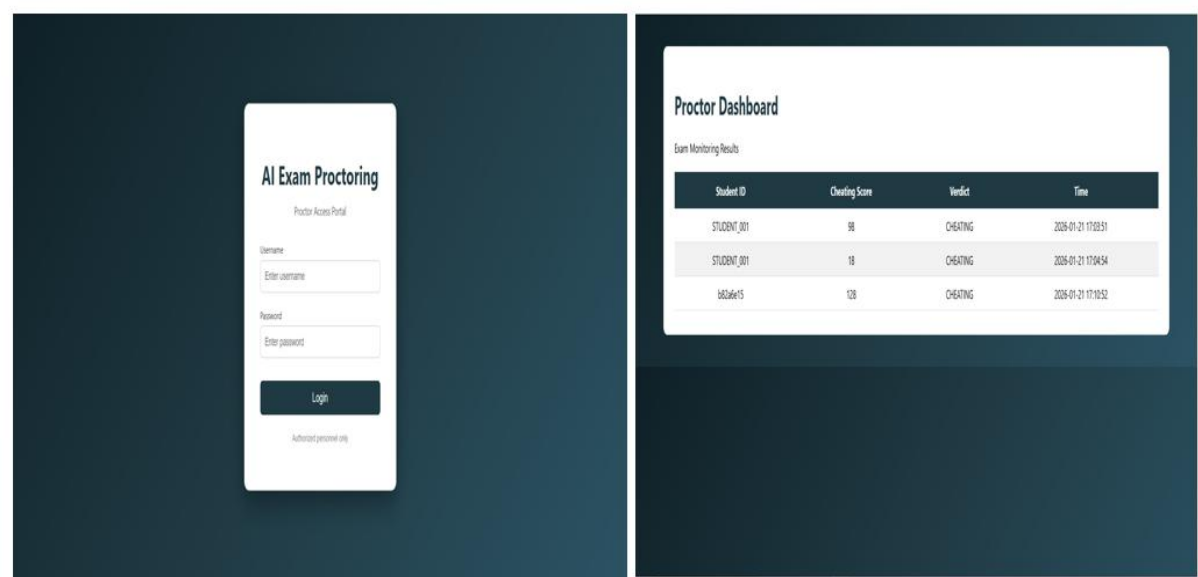


Figure-8

7. CONCLUSION AND FUTURE SCOPE

7.CONCLUSION AND FUTURE SCOPE

The AI-Based Intelligent Online Exam Proctoring System provides an effective solution to maintain academic integrity in online examinations. By automating the detection of suspicious behavior and storing exam results for proctor review, the system reduces human effort and improves reliability.

The project demonstrates the practical application of artificial intelligence in education and can be further enhanced with advanced features such as eye tracking and object detection.

FUTURE SCOPE:

The future scope of the AI-based proctoring system includes enhancing monitoring accuracy by integrating advanced deep learning models for eye movement tracking, head pose estimation, and emotion detection. Audio monitoring can be added to detect suspicious conversations or background voices during exams. Object detection techniques can be implemented to identify unauthorized materials such as mobile phones or books. The system can also be improved by incorporating browser activity and screen monitoring to prevent tab switching or external searches. Deploying the platform on cloud infrastructure would allow better scalability and remote access. Real-time alerts can be introduced to notify proctors instantly when suspicious behavior is detected. Additionally, facial recognition can be integrated for continuous identity verification, and advanced analytics dashboards can be developed to provide detailed behavioral and performance reports for institutions.

8.REFERENCES

8.REFERENCES

- [1] P. Viola, M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," IEEE Conference on Computer Vision and Pattern Recognition, 2001.
- [2] G. Bradski, A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library, O'Reilly Media, 2008.
- [3] OpenCV Documentation, "Open Source Computer Vision Library," Available: <https://opencv.org>